

The U-boot bootloader



U-Boot

U-Boot is a typical free software project

- License: GPLv2 (same as Linux)
- Freely available at: https://u-boot-project.org/
- Documentation available at: https://docs.u-boot-project.org/
- The latest development source code is available in a Git repository: https://git.u-boot-project.org/u-boot/u-boot.git
- Development and discussions happen around an open mailing-list: https://lists.u-boot-project.org/pipermail/u-boot/
- Follows a regular release schedule. Every 2 or 3 months, a new version is released. Versions are named `YYYY.MM`.

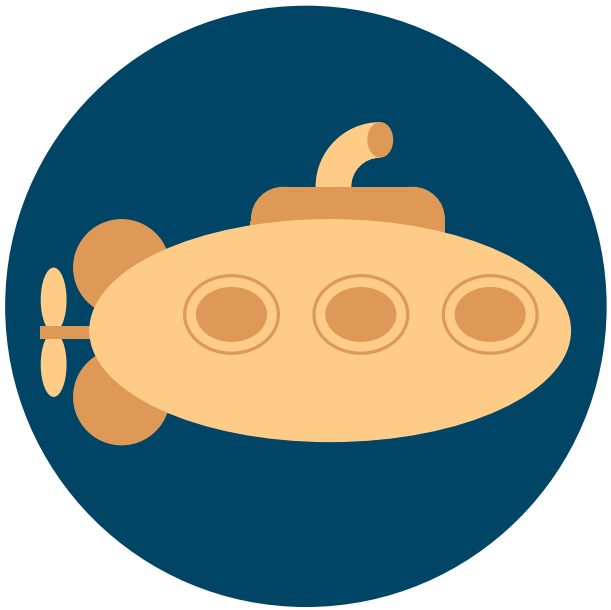


Image source



Where to get U-Boot from?

- **Ideal:** your platform is supported directly by **upstream** U-Boot
  - Best quality → code reviewed and approved by the community
  - Long-term maintenance
  - Use directly U-Boot from https://git.u-boot-project.org/u-boot/u-boot.git Git repository
- **Less ideal:** use a **fork** of U-Boot by your silicon vendor, system-on-module vendor or board vendor
  - Generally older, does not follow all upstream U-Boot updates
  - Changes not reviewed by the community → quality is often dubious
  - Check your HW vendor documentation/SDK
- If designing your own custom board
  - You will have to port U-Boot
  - If good support for your SoC in upstream U-Boot → use upstream U-Boot
  - If not → use the U-Boot fork from your SoC vendor



U-Boot configuration

- Configuration system based on *kconfig* from the Linux kernel
- The configs/ directory contains configuration files for supported boards or platforms
  - There may be a single configuration supporting multiple boards based on the same processor
  - The configuration files defines all relevant options: CPU type, drivers needed, U-Boot features to compile in
  - Examples:
    - configs/stm32mp15_basic_defconfig
    - configs/stm32mp15_trusted_defconfig
- Note: migration to *kconfig* is still on-going
  - Not all boards have been converted to the new configuration system.
  - Many boards still have a combination of configuration settings in include/configs/ header files, and configuration settings in `defconfig` files



U-Boot configuration file: stm32mp15_trusted_defconfig

```
CONFIG_ARM=y
CONFIG_ARCH_STM32MP=y
CONFIG_TFABOOT=y
CONFIG_SYS_MALLOC_F_LEN=0x3000
CONFIG_ENV_OFFSET=0x280000
CONFIG_ENV_SECT_SIZE=0x40000
CONFIG_DEFAULT_DEVICE_TREE="stm32mp157c-ev1"
[..]
CONFIG_CMD_ADTIMG=y
CONFIG_CMD_ERASEENV=y
CONFIG_CMD_NVEDIT_EFI=y
CONFIG_CMD_MEMINFO=y
CONFIG_CMD_MEMTEST=y
CONFIG_CMD_UNZIP=y
CONFIG_CMD_ADC=y
CONFIG_CMD_CLK=y
CONFIG_CMD_DFU=y
CONFIG_CMD_FUSE=y
CONFIG_CMD_GPIO=y
[...]
CONFIG_SPI=y
CONFIG_DM_SPI=y
CONFIG_STM32_QSPI=y
CONFIG_STM32_SPI=y
[...]
```

See the full file: configs/stm32mp15_trusted_defconfig

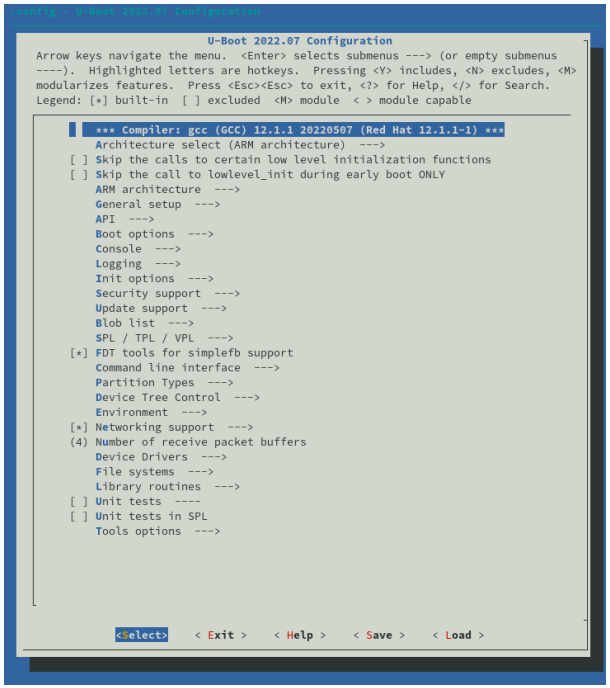


U-Boot configuration

- U-Boot must be configured before being compiled
- Configuration stored in a `.config` file
- Load a pre-defined configuration

```
$ make BOARDNAME_defconfig
```

  Where `BOARDNAME` is the name of a configuration, as visible in the configs/ directory.
- You can then run `make menuconfig` to further customize U-Boot's configuration.





U-Boot compilation

- The path to the cross-compiler must be specified in the `CROSS_COMPILE` variable
- `CROSS_COMPILE` contains the prefix common to all cross-compilation tools, e.g `arm-linux-`
- Common to add the cross-compiler location in `PATH` to keep the `CROSS_COMPILE` value short

```
$ export PATH=/path/to/toolchain/bin:$PATH
$ make CROSS_COMPILE=arm-linux-
```

- The main result is a `u-boot.bin` file, which is the U-Boot image.
- Depending on your specific platform, or what storage device you're booting from (NAND or MMC), there may be other specialized images: `u-boot.img`, `u-boot.kwb`...



Concept of U-Boot SPL

- To meet the need of a two-stage boot process, U-Boot has the concept of *U-Boot SPL*
- SPL = *Secondary Program Loader*
- The SPL is a stripped-down version of U-Boot, made small enough to meet the size constraints of a first stage bootloader
- Configured through *menuconfig*, one can define the subset of drivers to include
- No U-Boot shell/commands: the behavior is hardcoded in C code
- For some platforms: TPL, *Tertiary Program Loader*, an even more minimal first stage bootloader to do TPL → SPL → main U-Boot.



Device Tree in U-Boot

- The *Device Tree* is a data structure that describes the topology of the hardware
- Allows software to know which hardware peripherals are available and how they are connected to the system
- Initially mainly used by Linux, now also used by U-Boot, Barebox, TF-A, etc.
- Used by U-Boot on most platforms.
- Since v2024.07 the Device Tree files location depends on `CONFIG_OF_UPSTREAM`:
  - `dts/upstream/src/ARCH/VENDOR` when `CONFIG_OF_UPSTREAM` is set
  - `arch/ARCH/dts` otherwise
- One `.dts` for each board: need to create one if you build a custom board
- U-Boot *defconfigs* usually specify a default Device Tree, but it can be overridden using the `DEVICE_TREE` variable
- More details on the *Device Tree* later in this course.



U-Boot build example: TI AM335x BeagleBoneBlack wireless

- One *defconfig* file suitable for all AM335x platforms: configs/am335x_evm_defconfig
  - Yes its name looks like it supports only the EVM (EValuation Module) board
  - Contains `CONFIG_DEFAULT_DEVICE_TREE="am335x-evm"` → uses arch/arm/dts/am335x-evm.dts by default
- One *Device Tree* file describing the BeagleBoneBlack Wireless: arch/arm/dts/am335x-boneblack-wireless.dts
- Configure and build U-Boot

```
$ export PATH=/path/to/toolchain/bin:$PATH
$ make am335x_evm_defconfig
$ make DEVICE_TREE=am335x-boneblack-wireless CROSS_COMPILE=arm-linux-
```

- Produces:
  - `MLO`, the SPL, first-stage bootloader. Called MLO (*Mmc LOad*) as required on TI platforms.
  - `u-boot.img`, full U-Boot, second-stage bootloader



Installing U-Boot

1. If U-Boot is loaded from external storage, just update the binaries on such storage.
2. If U-Boot is loaded from internal storage (eMMC or NAND), you can update it using *Snagboot* (https://github.com/bootlin/snagboot) if it supports your SoC, or with the custom solution from the SoC vendor.
3. An alternative is to reflash internal storage with JTAG (if available), but that's more complicated and requires a JTAG probe.



U-boot shell prompt

- Connect the target to the host through a serial console.
- Power-up the board. On the serial console, you should see U-Boot starting up.
- The U-Boot shell offers a set of commands.
- The U-Boot shell is not a Linux shell: commands are completely different from Linux ones.

```
U-Boot SPL 2022.01 (Mar 31 2022 - 14:58:17 +0200)
Trying to boot from MMC1

U-Boot 2022.01 (Mar 31 2022 - 14:58:17 +0200)

CPU  : AM335X-GP rev 2.1
Model: TI AM335x BeagleBone Black DRAM:  512 MiB
WDT:   Started wdt@44e35000 with servicing (60s timeout)
NAND:  0 MiB
MMC:   OMAP SD/MMC: 0, OMAP SD/MMC: 1
Loading Environment from FAT... OK
Net:   Could not get PHY for ethernet@4a100000: addr 0
eth2: ethernet@4a100000, eth3: usb_ether [PRIME]
Hit any key to stop autoboot:  0
=>
```



U-Boot `help` command

- `help` command to list all available commands
- The set of available commands depend on the U-Boot configuration
  - Many `CONFIG_CMD_*` options to enable commands at compile time
  - See *Command line interface* submenu in `menuconfig`
- `help <command>` for the complete help of one command

```
STM32MP> help
?          - alias for 'help'
adc        - ADC sub-system
adtimg     - manipulate dtb/dtbo Android image
base       - print or set address offset
[...]
usb        - USB sub-system
[...]

STM32MP> help usb
usb - USB sub-system

Usage:
usb start - start (scan) USB controller
usb reset - reset (rescan) USB controller
usb stop [f] - stop USB [f]=force stop
usb tree - show USB device tree
usb info [dev] - show available USB devices
[...]
```



U-Boot information commands

Version details: `version`

```
=> version
U-Boot 2020.04 (May 26 2020 - 16:05:43 +0200)
arm-linux-gcc (crosstool-NG 1.24.0.105_5659366) 9.2.0
GNU ld (crosstool-NG 1.24.0.105_5659366) 2.34
```

NAND flash information: `nand info`

```
=> nand info
Device 0: nand0, sector size 128 KiB
  Page size       2048 b
  OOB size          64 b
  Erase size    131072 b
  subpagesize     2048 b
  options     0x40004200
  bbt options 0x00008000
```

MMC information: `mmc info`

```
=> mmc info
Device: STM32 SD/MMC
Manufacturer ID: 3
[...]
Capacity: 14.8 GiB
Bus Width: 4-bit
```

Board information: `bdinfo`

```
=> bdinfo
boot_params = 0x00000000
DRAM bank   = 0x00000000
-> start    = 0xc0000000
-> size     = 0x20000000
flashstart  = 0x00000000
flashsize   = 0x00000000
flashoffset = 0x00000000
baudrate    = 115200 bps
relocaddr   = 0xddb21000
reloc off   = 0x1da21000
[...]
fdt_blob    = 0xdbb01950
new_fdt     = 0xdbb01950
fdt_size    = 0x0001d540
Video       = display-controller@5a001000 inactive
[...]
```

- DRAM starts at `0xc0000000`, for a size of 512 MB (`0x20000000`).
- The end of the memory is used by U-Boot itself: `relocaddr` is the location of U-Boot in RAM.



Concept of U-Boot environment

- A significant part of the U-Boot configuration happens at compile time: `menuconfig`
- U-Boot also has runtime configuration, through the concept of *environment variables*
- Environment variables are key/value pairs
  - Some specific environment variables impact the behavior of different U-Boot commands
  - Additional custom environment variables can be added, and used in *scripts*
- U-Boot environment variables are loaded and modified in RAM
- U-Boot has a default environment built into its binary
  - used when no other environment is found
  - defined in the configuration
  - the default environment is sometimes quite complex in some existing configurations
- The environment can be persistently stored in non-volatile storage



U-Boot environment persistent storage

Depending on the configuration, the U-Boot environment can be:

- At a fixed offset in NAND flash
- At a fixed offset on MMC or USB storage, before the beginning of the first partition.
- In a file on a FAT or ext4 partition
- In a UBI volume
- Not stored at all, only the built-in environment in the U-Boot binary is used

```
.config - U-Boot 2020.07 Configuration
 Environment
[ ] Environment is not stored
[ ] Environment in EEPROM
[*] Environment is in a FAT filesystem
[ ] Environment is in a EXT4 filesystem
[ ] Environment in flash memory
[ ] Environment in an MMC device
[ ] Environment in a NAND device
[ ] Environment in a non-volatile RAM
[ ] Environment is in OneNAND
[ ] Environment is in remote memory space
[ ] Environment in a UBI volume
    (mmc) Name of the block device for the environment
    (0) Device and partition for where to store the environemt in FAT
    (uboot.env) Name of the FAT file to use for the environment
    (0x4000) Environment Size
[*] Relocate gd->env_addr
[ ] Create default environment from file
[ ] Add run-time information to the environment
[ ] Block forced environment operations
```

U-Boot environment configuration menu



U-Boot environment commands

- `printenv`
  Shows all variables
- `printenv <variable-name>`
  Shows the value of a variable
- `setenv <variable-name> <variable-value>`
  Changes the value of a variable or defines a new one, only in RAM
- `editenv <variable-name>`
  Interactively edits the value of a variable, only in RAM
- After an `editenv` or `setenv`, changes in the environment are lost if they are not saved persistently
- `saveenv`
  Saves the current state of the environment to storage for persistence.
- `env` command, with many sub-commands:
  `env default`, `env info`, `env erase`, `env set`, `env save`, etc.



U-Boot environment commands example

```
=> printenv
baudrate=19200
ethaddr=00:40:95:36:35:33
netmask=255.255.255.0
ipaddr=10.0.0.11
serverip=10.0.0.1
stdin=serial
stdout=serial
stderr=serial
=> setenv serverip 10.0.0.100
=> printenv serverip
serverip=10.0.0.100
=> saveenv
```